# ShopTrack
## Simple Order Management with CouchDB and MySQL

Final Group Project | Project Proposal

**Group 1: Hassan Alshamrani, Damian Mendoza, Ingri Quevedo, Stephanie Armstrong**

**Assigned NoSQL database:** Apache CouchDB

**Primary database model:** Document database (stores data as JSON documents and is used through HTTP)

## 1. Business Problem and Users

ShopTrack is a simple order management project for a small online store. The store needs an easy way to keep track of its orders. **Store managers** want to see which orders are processing, shipped, delivered, or cancelled. **Customer support staff** need to find a customer's orders quickly and update an order when something changes.

This problem fits CouchDB well because each order is one complete record: the customer, the products, the prices, the address, and the dates. In CouchDB, the whole order can be saved as one JSON document, so we can get everything about an order in one step. We will load the same data into MySQL so we can compare the two databases on the same tasks.

## 2. Dataset

We will use **dummy (synthetic) data**, not real customer purchases. The data structure and product examples are based on a public sample orders dataset (JSONSift), and all orders were generated for practice. The dataset files will be submitted separately.

- **Size:** 500 orders, 80 customer IDs, 8 products, 1,520 order items, and 5 cities (Austin, Boston, Chicago, San Francisco, Seattle).
- **Order statuses:** processing, shipped, delivered, and cancelled (125 orders each).
- **Format:** one JSON file for CouchDB and one SQL file for MySQL. Both files contain the same orders.
- **Preparation:** the files are already prepared. After importing, we will check that both databases show 500 orders. We have not tested the imports yet.

**Why this data fits:** each order has a list of items inside it, which suits a document database. The same data also fits naturally into MySQL tables, so it is a fair way to compare the two. It also meets the course guideline of at least 500 records, and since it is dummy data, there are no privacy concerns.

## 3. Data Fields

Each order in the JSON file has these fields:

| Field | Meaning |
|---|---|
| id | Order number, e.g., syn_ord_0001 (CouchDB also uses it as the document ID, _id) |
| userId | Customer ID, e.g., syn_usr_001 |
| status | processing, shipped, delivered, or cancelled |
| items | List of products in the order. Each item has: sku (product code), name, quantity, and unitPrice |
| subtotal | Price of all items before tax and shipping |
| tax | Tax amount |
| shipping | Shipping cost |
| total | Final amount (subtotal + tax + shipping) |
| shippingAddress | Delivery address: line1, city, state, zip, and country (all fictional) |
| createdAt | Date and time the order was placed |
| deliveredAt | Date and time the order was delivered (empty if not delivered) |
| currency | Currency of the prices (USD) |
| dataOrigin | A label showing the record is dummy data |

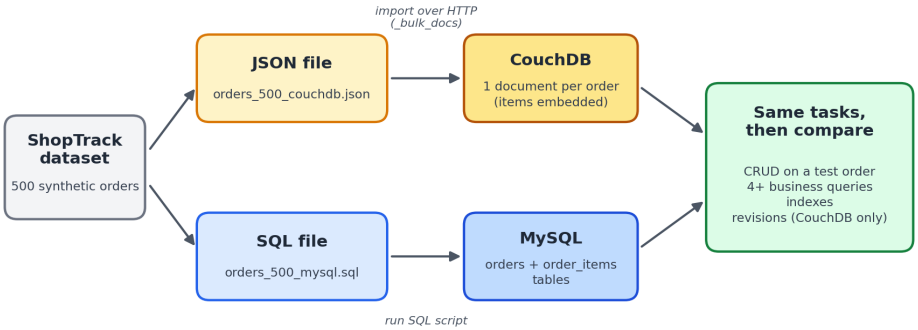
In MySQL, the same fields are used with slightly different names (for example, userId becomes user_id).

## 4. Preliminary Data Model

- **CouchDB:** one database with one document per order. The items and the address are kept inside the order document, because they are always viewed together with the order.
- **MySQL:** two tables. The orders table has one row per order, and the order_items table has one row per item. The two tables are linked by the order ID (primary key and foreign key).

This model may change after we start working with the databases.

## 5. Architecture / Data Flow




*The same data goes into CouchDB and MySQL, then we compare the results*

## 6. What We Plan to Do

1. Install CouchDB and MySQL and import the data.
2. Add a test order, display an order, update an order's status, and delete the test order.
3. Run at least four business queries, such as: delivered orders with a total over $200; processing orders created before a chosen date; one customer's orders within a date range; and shipped orders going to a selected city.
4. Create indexes to help these queries.
5. Show a CouchDB feature: document revisions, which stop two people from overwriting each other's changes.
6. Do the same tasks in MySQL and compare the results.

## 7. Preliminary Comparison with MySQL

| Topic | CouchDB | MySQL |
|---|---|---|
| How an order is stored | One JSON document with its items inside | Split into two tables: orders and order_items |
| Getting a full order | One request by order ID | Join the two tables |
| Structure rules | Flexible; documents can be different | Fixed table structure with keys |
| Handling changes | Each document has a revision number | Transactions and locking |

*Table 2. Early comparison (to be tested)*

We expect CouchDB to be easier for saving and reading complete orders, and MySQL to be stronger for reports across many orders. We will give our final recommendation after testing. In the final report, we will also compare CouchDB with **MongoDB**, a NoSQL document database covered in class.

## 8. Team Responsibilities

The table is listed in the order we plan to present. Each member has a main task during the project, a part in the presentation, and a part in the live demo.

| Member | Main task (project work) | Presentation (slides) | Live demo |
|---|---|---|---|
| **1. Hassan Saleh H Alshamrani** | Business problem, dataset, and architecture diagram | Introduction, business problem, and dataset | Add a new order and display it |
| **2. Damian Mendoza** | Install CouchDB and MySQL, import the data, and set up the MySQL tables | Data model: CouchDB documents vs. MySQL tables | Show the loaded data; run the business queries in MySQL |
| **3. Ingri Quevedo** | Update and delete operations; CouchDB revisions | CouchDB features (revisions) | Update and delete an order; show how revisions prevent overwriting |
| **4. Stephanie Armstrong** | Business queries, indexes, and the MySQL and MongoDB comparisons | Comparisons and final recommendation | Run the business queries and indexes in CouchDB |

***These responsibilities are subject to change.*** *Every member will help with the whole project, understand all parts, and take part in the presentation and demo.*

## 9. Technical Risks

- **Installation problems:** team members use different computers, so setting up CouchDB and MySQL may not work the same way for everyone.
- **Import errors:** importing the files twice or into the wrong place could create duplicate or conflicting data. We will import once and check the counts.
- **Learning CouchDB queries:** CouchDB uses its own query style (Mango queries) and indexes, which are new to us, so writing the queries may take extra time.

## 10. Questions

1. Our MySQL file currently uses two tables (orders and order_items). Should we build a fully normalized MySQL design (for example, adding a products table), or is explaining it in the final report enough?
2. Is showing document revisions acceptable as our CouchDB-specific feature, or would you prefer that we show replication?